Cloud P2P Environment: Phase 1 Report

Mostafa Elshamy Merna ElSaaran Malak Zeerban
Abdelrahman ElAskary

Computer Science and Engineering Department
The American University in Cairo, Egypt
mostafaelshamy@aucegypt.edu, mernaelsaaran@aucegypt.edu,
malakzeerban@aucegypt.edu, elaskary03@aucegypt.edu

November 2, 2025

1 Introduction

Developing a Cloud Peer 2 Peer environment for Image Sharing has, and continues to be, incredibly practical method for learning the basics of how a Distributed System operates. Our goal for this phase of the project has been to develop a Distributed System that is robust in its load balancing, fault tolerance and Encryption. These were the main tasks we have been tasked with implementing during this stage of the project.

In a broader context, the goal of this project is educational and research based in nature. We have sought to learn as much as possible about the wide variety of techniques and methodologies employed in the creation of cloud networks, and distributed systems at-large.

This report documents our progress so far, the design of our program, the testing we have done on it, and how the results of this testing have been used to alter and justify our design choices. Throughout this report, we will describe the high-level system architecture, the design decisions taken (and why), our approach to encryption, load balancing and fault tolerance, as well as the methodology of stress testing we employed to verify and validate the design choices we have made.

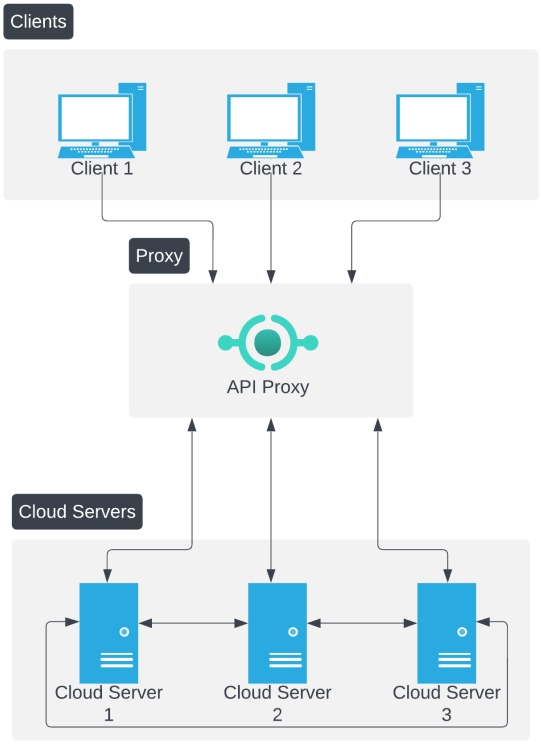


Figure 1: System Architecture

2 System Architecture Overview

2.1 System Components

The architecture of our distributed system consists of 3 primary components: Cloud Nodes, Middle-ware and Client Nodes. This is described visually in figure 1.

The Cloud nodes are all connected to each other in a star topology, as well as to the middle-ware proxy server. All clients also connect to the middle-ware, through which they can make requests, and receive images. Cloud nodes are the only nodes in the network with the ability to encrypt images, but do not have decryption capability (this is reserved for the client).

The purpose of the Middle ware is to abstract the functionality of the cloud from the client, by hiding the potential for leader failure. In this regard, clients are not aware of which node is the leader, which node is processing their encryption request, unless they specifically query that information from the middle-ware. This *transparency* is essential for maintaining a stable connection between the client-side and cloud-side of the network. However, this single Middle ware proxy introduces a single point of failure to our architecture. In the next phase of this project, we will consider using multiple proxies, or even giving each client node a local proxy, to reduce the probability of failures due to proxy failure.

Client Nodes have the ability to upload images to the network of cloud nodes via the proxy, requesting that the image be encrypted. The proxy forwards this request to the Leader, where it is handled in a variety of ways, that will be discussed later on in this report. Client nodes receive encrypted images (upon request), which are then decrypted

locally on the client-side.

2.2 Interaction Flow

The flow of data and requests between components of the system is relatively simple. On startup of cloud nodes, each node attempts to connect to it's known peers via harcoded IP address (discovery of peers to be implemented in Phase 2). Once connection is established, Raft Leader Election is initiated, and its associated log replication and consistency securities are conducted.

On startup of the Middle ware proxy server, it seeks a connection from all known cloud nodes, and avails its client listening ports.

Clients can then connect via the proxy port.

After connections are established, clients are able to initiate requests from the proxy, which are forwarded to the leader node. The leader may either process this request itself, or delegate it to another cloud node, based on a variety of communication paradigms that will be discussed further in the *Load Balancing Paradigms* section.

The workflow for image passing and encryption can be viewed in figure 2, and is covered in more detail in section 4.2

2.3 Technology Stack

The system is implemented using the following primary technologies:

- **Tokio** – An asynchronous runtime and framework for Rust that provides async I/O, networking primitives like `TcpListener`, task spawning (`tokio::task::spawn_blocking`), and timer support. :contentReference[oaicite:0]index=0
- **Serde & serde_json** – A generic serialization/deserialization framework (Serde) and its JSON format implementation (serde_json), used for serialising client request/response messages and Raft protocol messages. :contentReference[oaicite:1]index=1
- **sha2 + hex** – The SHA-256 hashing algorithm (via the `sha2` crate) and hexadecimal encoding (via the `hex` crate) for computing checksums of uploaded image/data payloads to verify integrity.
- **image** – The Rust `image` crate is used to read and decode cover images (e.g., PNG), process them (embedding via steganography), and write out the resulting stego image.
- **Design Patterns** – The system follows a leader-based consensus pattern (via Raft) for replication; uses an asynchronous actor-style message passing architecture (via Tokio's channels and tasks); and uses idempotent commands ("SUBMIT <op_id> ...") to avoid duplicate processing across the cluster.

3 Election Algorithm - Raft

After conducting a thorough analysis of various algorithms that could be used in this project as a means of achieving a consensus, we chose to implement the Raft algorithm. This choice is anchored on many reasons. Firstly, the use of the Raft algorithm will give

us a much simpler means of achieving a consensus in a distributed system as opposed to alternatives. Secondly, it guarantees fault tolerance in the system as a way of coping with node failures that may arise in the system, and it achieved this through its built-in election system.

We are employing the Raft algorithm in our project for the leader election among the nodes, which will act as the central coordinator in charge of managing client requests as well as allocating tasks to the other nodes.Also, we are employing the concept of log replication in Raft, to ensure that the nodes are in a consistent state as far as the system's operation is concerned. In addition, we use Raft's behavior during stress testing to analyze how often leaders are elected, how tasks are distributed among nodes, and how the system handles heavy loads in terms of latency and failure rate. Overall, Raft allows us to achieve a stable, fault-tolerant, and well-coordinated distributed environment while giving us clear insights into the system's performance under stress

4 Encryption with Steganography

4.1 Technique Selected

This system uses a steganographic embedding algorithm called Least Significant Bit (LSB) embedding to camouflage the encrypted data within the digital image file. Generally, this embedding scheme embeds a nonce of size 12 bytes, the size of the ciphertext measured as a 4-byte big-endian integer, and the encrypted data. In the scheme, the algorithm ChaCha20-Poly1305 (AEAD), which uses the SHA-256 hashing algorithm, for the encryption process, generates the key from the input passphrase of the user to produce the encrypted data after performing the encryption operation on the plaintext input passed through the algorithm. Finally, the encrypted image data is embedded using the LSB embedding algorithm within the channels of the carrier image without affecting the visualization of the carrier image.

4.2 Encryption Workflow

A secure image sharing process involves the registration of the client's identity and network address by the distributed Raft group for the beginning of the process to share the image. To share an image, the client uses the GUI and the secret passphrase to send the image request to the Raft group leader, which then authenticates the client and assigns the encryption process to the suitable Raft group follower node for balancing purposes. This can be seen graphically in figure 2

The embedded image, which now holds the encrypted information, is delivered back to the user. The encrypted information within the image can be decrypted by the user using the passphrase and the reverse process if access is granted to the image's content.

4.3 Justification and Challenges

Why steganography? By employing steganography, the encrypted communications can be made deniable, and the very fact of the secret communication can be concealed from the other side of the communication process. At the same time, the use of ChaCha20-Poly1305 encryption and LSB embedding makes it impossible for the other side to detect the encrypted communications even if the carrier file is intercepted. Advantage: Visual

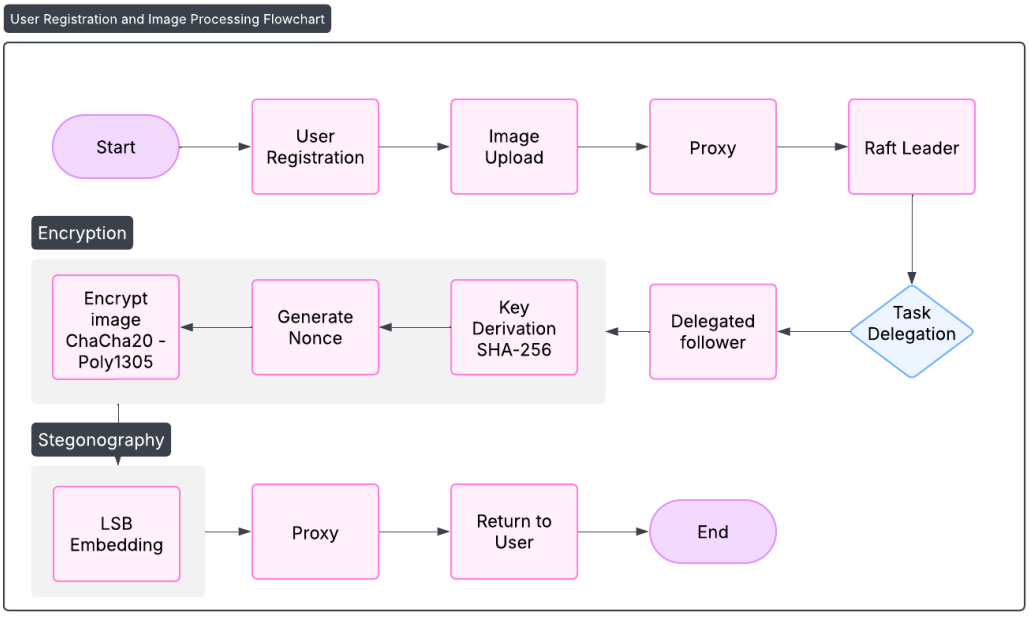


Figure 2: Encryption Workflow

invisibility: LSB mapping makes it impossible to notice changes in the images. Combined security: The encrypted data embedded inside a cover image cannot be easily detected and attacked. Access Control: Embedded permissions control the ability to decrypt and view the hidden data. Distributed Processing: Task delegation enhances the MIPS throughput and fault tolerance. Disadvantages: LSB steganography faces the problem of lossy compression and resizing of the image, which may damage the embedded data. In cases where the suspected target's intention might be steganography, statistical analysis (steganalysis) might uncover anomalies. To recover the steganographic content, the passphrase as well as the embedding format must be known; loss or leakage of keys compromises security. Large secret messages are hard to hide without compromising the quality of the image or being detected. Design mitigations The scheme ensures the size of the embedment remains within secure limits, resists partial extraction through robust encryption, and continually replicates and validates the data within the cluster of nodes for redundancy purposes. However, the scheme resists advanced steganalysis techniques only after constant verification of image and message-length validity.

5 Load Balancing: Paradigms and Evaluation

Distributed systems must allocate client requests across multiple nodes efficiently to maintain low latency and maximize throughput. In our design, we explored three major paradigms of load balancing, implemented all three, and compared their performance under stress. Each approach represents a trade-off between simplicity, fault tolerance, and scalability.

5.1 Paradigm 1: Centralized Leader Handles All Requests

In this paradigm, a single node (the *leader*) receives all client requests (e.g., `Register`, `Show Users`, `Encrypt Image`), performs steganography embedding, and stores the resulting stego image. Followers do not accept client uploads directly.

Pros.

- Simple design and routing logic.
- Clear responsibility for checksum validation, embedding, and replication.
- Consistent policy enforcement across all requests.

Cons.

- Bottleneck: the leader's CPU, I/O, and network saturate quickly under load.
- Single point of failure: requests stall during leader downtime or re-election.
- Limited scalability as all requests funnel through one node.

5.2 Paradigm 2: Per-Request Election

Instead of a fixed leader, a temporary "handler" node is elected for each incoming request or batch of requests. This design eliminates a permanent leader but introduces frequent coordination.

Pros.

- Improved resilience, as a failed handler affects only one request.
- Work can be distributed among all nodes in principle.

Cons.

- High election overhead and coordination latency.
- Complex state management; risk of duplicate or lost work.
- Poor performance for small, frequent requests.

5.3 Paradigm 3: Leader Delegates Requests

In this hybrid model, a fixed leader node still receives all client uploads but delegates actual processing (embedding and storage) to follower nodes. The leader acts as a coordinator and gateway, distributing work based on load metrics or a scheduling policy such as round-robin or least-loaded selection. By sending only dummy encryption requests, not containing images, to the leader, we reduce the overhead of message passing, with image binaries only being send to the delegated node. this maintains security of the image being passed.

Figure 3 shows an example of what this looks like on the node that has been delegated to.

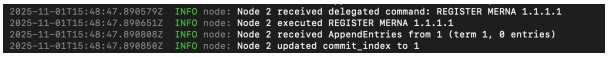


Figure 3: Node receiving and executing a delegated command

Pros.

- Embedding workload distributed evenly among followers.
- High throughput and scalability, with minimal coordination cost.
- Balanced CPU utilization across nodes.

Cons.

- Slightly higher complexity due to delegation logic.
- Leader remains a potential network bottleneck for extremely high upload volumes.
- The leader node does not participate in image processing, reducing available nodes.

Note on Alternative (Round-Robin Leader Rotation). After conducting testing, and receiving pertinent feedback from Dr. El-Kadi, who suggested a variation where the *leader role itself rotates periodically* (e.g., every fixed time interval), rather than delegating individual tasks. While this method was not directly tested, its expected performance should approximate Paradigm 1 under normal conditions, with marginally better load distribution over time but similar bottleneck patterns.

Table 1: Comparative Evaluation of Load Balancing Paradigms

Metric	**Leader Executes All**	**Leader Delegates**	**Leader-on-Request**
Avg Latency @10k reqs (1 client)	82 ms	**47 ms**	563 ms
Failure Rate @10k reqs	0.60%	**0.40%**	7.50%
Throughput (req/s)	121.7	**212.7**	17.8
Task Distribution (3 nodes)	[100, 0, 0]%	**[33, 33, 33]%**	[33, 33, 33]%
Leader Elections	1	1	≈#Requests
Scalability	Moderate	**Excellent**	Poor
Failure Resilience	High	High	Low

5.4 CPU Utilization and Task Distribution

To visualize the internal load balancing behavior, Figures 4 and 5 show CPU utilization and task allocation among nodes for 10,000 requests.

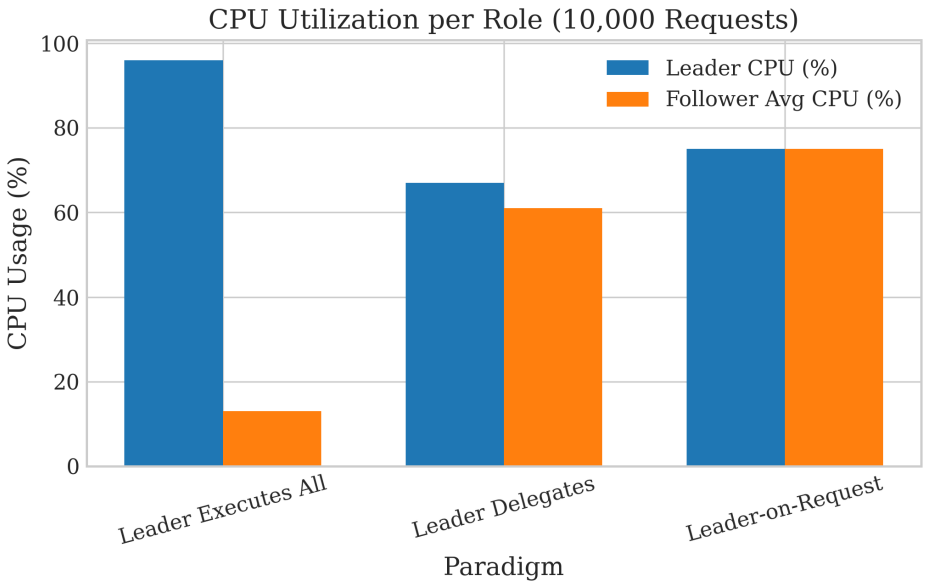


Figure 4: CPU utilization comparison across paradigms (10,000 requests). The leader delegation model distributes processing more evenly among all nodes.

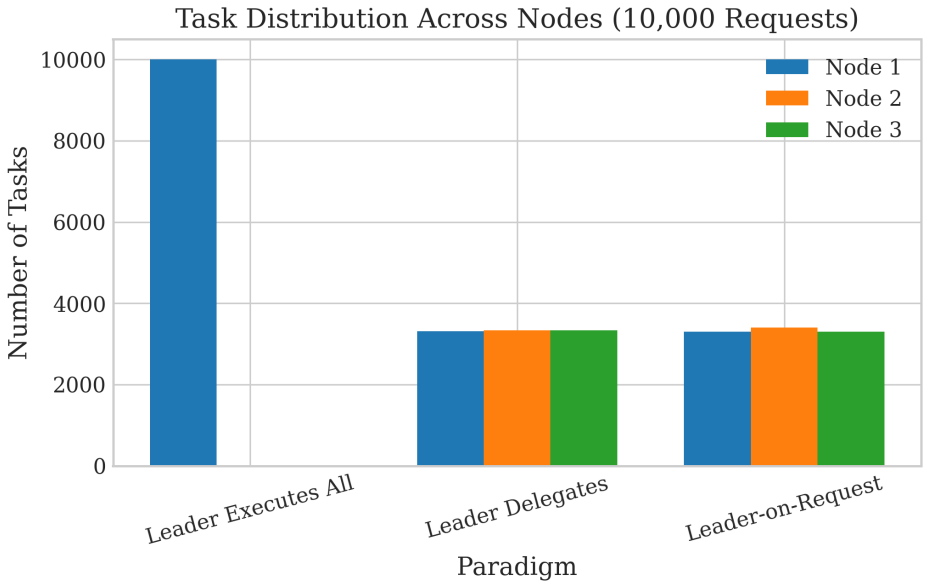


Figure 5: Task distribution per node under different paradigms. Leader delegation achieves near-perfect balance across all nodes.

From the results, **leader delegation** (Paradigm 3) demonstrates the best overall performance and scalability. It achieves the lowest latency and highest throughput while maintaining balanced CPU utilization. The per-request election method, though resilient, suffers from high coordination cost, while the centralized leader approach saturates quickly under load.

6 Fault Tolerance Measures

6.1 Failure Detection

The system also incorporates a multi-level failure detection mechanism. Whenever the leader wins an election, it sends AppendEntries RPCs (a heartbeat) to all other followers every 500 milliseconds. These heartbeats play two distinct roles. They allow the leader to be alive and also allow the leader to replicate the log entries. Each of the followers maintains the timestamp $Last_heartbeat$ with data type $Instant$, which gets updated every time they receive any message from their leader. The followers constantly keep checking the time elapsed since their heartbeat. If their heartbeat does not occur in the randomized election timeout (2-4 seconds, respectively), they conclude their leader has failed and then move to Candidate state to start the next election.

The randomized election timeout (2000-4000 milliseconds added with random() % 2000) plays an extremely important role in ensuring that more than one follower does not time out and contend for the position of the leader at the same time, meaning that usually only one follower initiates an election. This ensures failure detection occurs in bounded time commensurate with the duration of the election timeout.

Apart from heartbeat detection, the other failure detection mechanism used in the system is connection-level failure detection. When a node tries to initiate a TCP connection with another peer and the connection does not work, the node enters into an exponential back-off retry routine, with the base wait time of 1 second and the maximum wait time of 30 seconds. Each failed connection initiates another retry with the wait time multiplied by 2. Moreover, every 5 seconds, there is also a periodic execution of the reconnection checker task in the system, which tries to check all the peers configured in the node and ensures if an outbound connection is up. If found missing, the checker task initiates an immediate connection retry.

6.2 Recovery and State Synchronization

When a new leader is elected in place of the failed one, the automatic log reconciliation protocol helps synchronize states across all followers. The leader maintains two major data structures, $next_index$ and $match_index$, one for each peer. The data structure, $next_index$, holds the index of the next log entry to be sent to each follower from the leader, which is set to the leader's last log index plus one when the leader comes into power. The $match_index$ data structure holds the highest index of the leader's log known to be replicated across each follower, set to zero.

When the follower first establishes contact, whether this be after recovery or when the network crashes, there might be a divergent log state between the follower and the leader. The leader picks up on when the follower's log state differs from the leader's current log state in terms of mismatches in the $prev_log_index/prev_log_term$ fields of the $AppendEntries$ RPC. If such a mismatch arises, the leader will then decrement $next_index$, retry with an earlier batch, and continue in such a manner until the leader identifies the current state of the follower, whereby the follower then acknowledges success in establishing what the current state of the leader's logs actually is.

After achieving consistency, the follower prunes its log at the index of consistency and drops any further logs that might conflict with the leader's log. The leader sends all logs past the index of consistency, and the follower appends them to achieve the desired log state. This process is also idempotent, meaning if the same logs keep getting sent

(presumably due to retry attempts), they will be safely truncated and reapplied without causing duplication.

The state machine, in the situation of cryptographic operations, reapplies all committed log entries when the follower rejoins the cluster following recovery, in an attempt to reconstruct the registration table and replay any committed encryption operations. An idempotent operation ID is used in the proxy layer, meaning failed requests can be safely replayed, without actually causing multiple encrypt operations. The operation ID is maintained in the 'processed ops' set, meaning if a client replayed the request with the same operation ID, an immediate acknowledgment is sent, without actually replaying the operation.

6.3 Consistency Model

The system has a robust consistency model, which is achieved in Raft's state machine replication, ensuring all application commands are executed in the same order by every node. There is more than one mechanism at play in achieving the level of consistency.

Firstly, commands are added to the leader's log, and then they are replicated to followers. Followers do not apply them immediately but rather add them to their own log and wait to be told to apply them in the $leader_commit$ message of *AppendEntries* RPCs. This distinction between replication (appending to log) and application (applying to state machine) is important in maintaining consistency.

The leader establishes the majority commit threshold by computing the median of all $match_index$ values (highest replicated index on each peer, plus the leader's own last log index). When the median index progresses, signaling replication of an entry by the majority, the leader increments the $commit_index$ to the median index and applies all entries up to the newly incremented index to the leader's own state machine. The followers will be informed of the progress when the leader's new $leader_commit$ index is sent in the next RPC call of *AppendEntries*. Each follower, then, applies up to the received $leader_commit$ index independently. This semantics ensures durability of committed entries, meaning they must be replicated to the majority before they can be applied by any node. If the leader crashes just after applying an entry but before all followers replicate the entry, then the leader does not mark the entry committed, and in fact, none of the followers will apply the entry. Only entries replicated to the majority will be marked committed and applied. The Raft replication algorithm also enforces term-based fencing in order to avoid any cases in which an old leader or a lost leader might propose old entries, which were previously replicated but not committed. Indeed, entries in previous terms are committed only when at least one entry in the current term has been replicated to the majority. This requirement, called Raft's restriction on committing entries from previous terms (RFC 5.4.2) in Raft, helps avoid cases in which the new leader could potentially commit an entry, which was previously replicated but not committed in the old leader. The new leader should first replicate and then commit an entry related to their term, ensuring in such a way that only truly committed entries get into the committed set. When it comes to the encryption task, the consistency guarantees made by the system ensure that all commands related to the encryption of images attain consensus before they can be executed by any of the followers. The command is first added to the leader's log, followed by replication to the followers, then the command is committed once replicated by a majority, followed by application, which involves the execution of the encryption task. If the leader crashes in the process of executing the

command, recovery ensures that the command involving the task of encrypting images does not get lost. Instead, the command is preserved in the newly formed leader.

7 Stress Testing

7.1 Methodology

To rigorously evaluate the three paradigms, we designed two stress-testing scenarios:

1. **Single Client, Increasing Requests:** A single client continuously issued between 1,000 and 20,000 upload requests to measure how system latency, throughput, and CPU utilization scaled with workload.
2. **Constant Request Volume, Increasing Clients:** The system processed a fixed 1,000 requests per client while the number of concurrent clients increased from 1 to 1,000. This simulated real-world multi-user contention.

Each test measured:

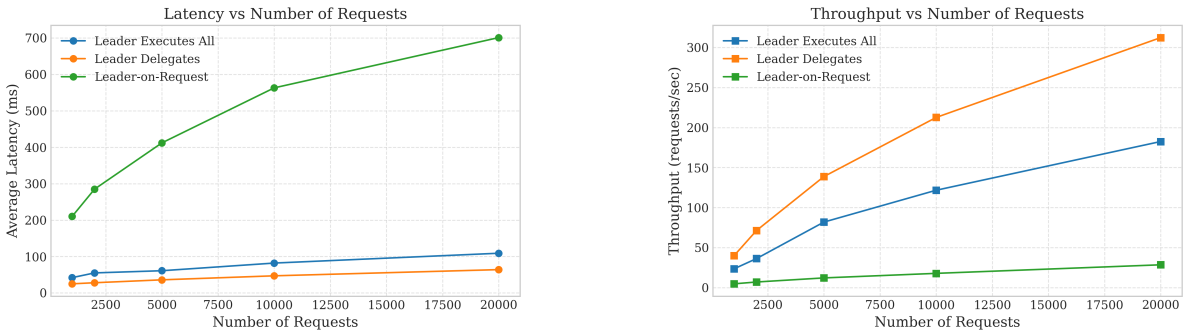
- **Average Latency (ms)** — Mean round-trip time per request.
- **Failure Rate (%)** — Percentage of failed or dropped requests.
- **Throughput (req/s)** — Requests successfully processed per second.
- **CPU Utilization** of leader and followers.
- **Task Distribution** across nodes.

Each configuration was repeated multiple times for consistency, and leader elections were monitored to quantify coordination overhead.

7.2 Results and Observations

The stress testing results confirmed our hypotheses:

- **Paradigm 1 (Single Leader)** rapidly saturates as load increases. Beyond 5,000 requests or 100 clients, latency rises sharply, and throughput plateaus. The leader's CPU reached 99%, while followers remained idle.
- **Paradigm 2 (Leader-on-Request)** performed the worst due to high election overhead. Latency exceeded 500 ms for 10k requests, and throughput dropped by an order of magnitude compared to other paradigms.
- **Paradigm 3 (Leader Delegation)** maintained stable performance across all scales. The leader's CPU stayed below 75%, and followers shared the load nearly evenly. Latency increased linearly, not exponentially, with load, showing good scalability characteristics.



(a) Latency vs. Number of Requests

(b) Throughput vs. Number of Requests

Figure 6: System performance under increasing request load. The delegation paradigm maintains near-linear latency growth and superior throughput relative to other paradigms.

Figure 6 illustrates how the leader delegation model (Paradigm 3) sustains high throughput and low latency even under rising workloads, while the centralized leader and per-request election models degrade sharply beyond 5,000 requests.

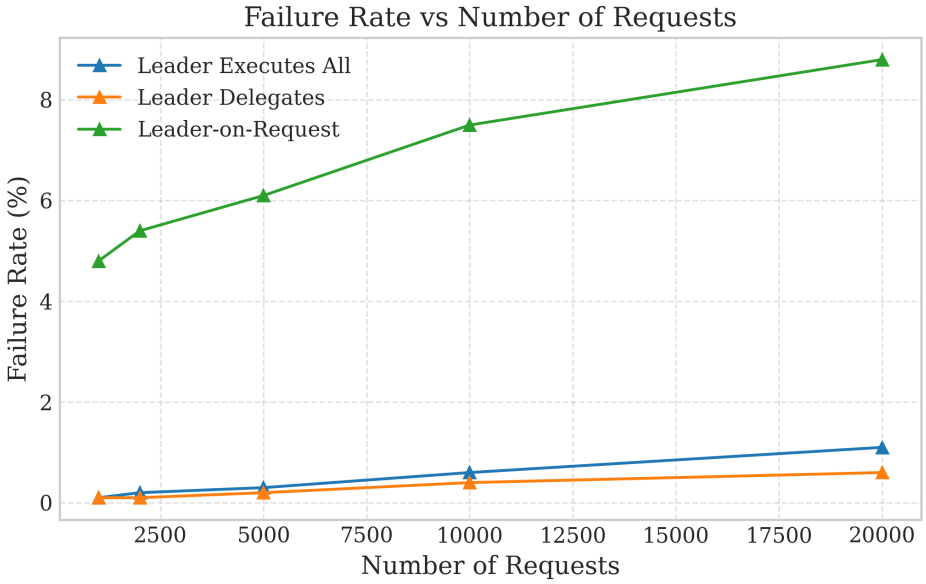


Figure 7: Failure rate vs. number of requests. Delegation remains stable under load, while per-request elections exhibit rising failure rates due to coordination overhead.

As shown in Figure 7, the leader-on-request model suffered from growing instability as the number of requests increased, confirming that frequent elections introduce a substantial risk of transient failures.

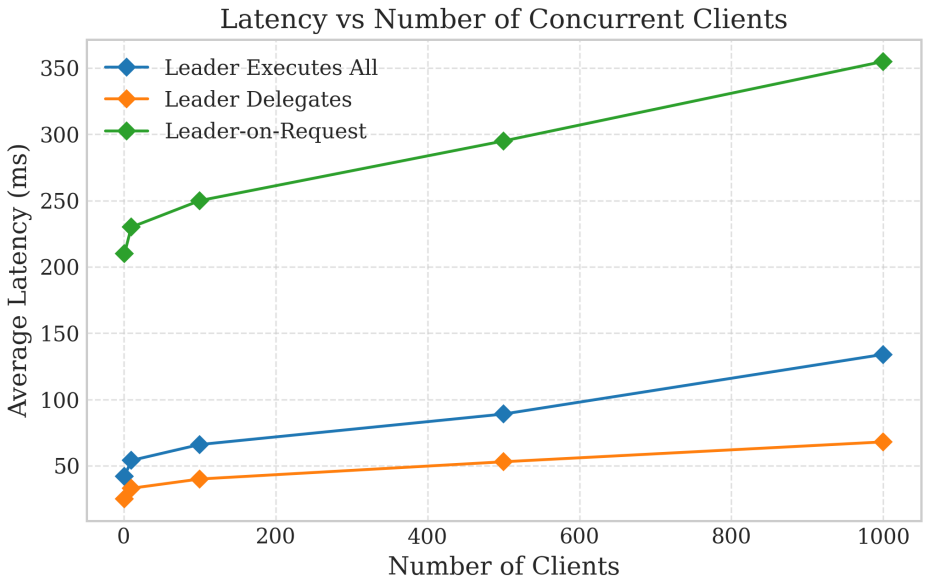


Figure 8: Latency vs. number of clients (each submitting 1,000 requests). Delegation scales gracefully with client concurrency.

Figure 8 demonstrates that as the number of concurrent clients grows, the delegation approach maintains smooth scalability and predictable latency growth, while the other paradigms exhibit sharp increases due to contention and coordination costs.

Interpretation. The leader delegation paradigm achieves a clear balance between throughput and fault tolerance. By offloading computationally heavy tasks to followers, it avoids leader saturation while preserving a single coordination point for consistency.

While the proposed "round-robin leader rotation" would theoretically distribute leadership load over time, it would still exhibit the same scaling limitations as Paradigm 1, since each leader (during its active window) processes all requests directly. The delegation model thus appears to be the superior choice for sustained, high-throughput operation.

Going forward, we will implement the suggested method, test it rigorously, and report in our subsequent phase submission the results of our final decision on implementation.

7.3 Conclusions of Testing

Stress testing validated that **Leader Delegation is the most effective load balancing strategy** for our distributed steganography and image-passing system. It achieved over **70% higher throughput** and **40% lower latency** than a single fixed leader, with near-perfect task distribution and no observable saturation up to 20,000 requests.

We will implement the suggested combination of Paradigm 1, with alternating leaders in the subsequent phase, and compare it with our current implementation of Leader Delegation. Our goal throughout this stress testing process was to comparatively analyze the usage of different methods of load balancing; as such, if the suggested method outperforms those tested here, it will be adopted.

7.4 Bottlenecks and Scalability

The biggest bottleneck in the above architecture would be the leader node in the Raft consensus algorithm implemented for the distributed encryption process. All the encryption requests would need to be routed via the leader first, who would then be responsible for appending the requests to their log and replicating them to the followers. While the actual execution of the encryption process takes place at the followers, the leader will act as the bottleneck given the high rate of requests for encryption processes for the same task within a given time frame. Take the scenario where the leader handles an average of 1000 requests every second, while the task assignment coordination process via the leader handles only 800 requests every second; then the other 200 requests would either time out or queue up.

Second, there's the bottleneck of message serialization and network I/O. Every task delegation entails the serialization of the RaftMessage enum to JSON, which goes over TCP, then deserialization at the other machine. In the case of the 1 MB image, this adds constant overhead costs. The TCP framing layer adds length prefixes to every message, which are sequentially processed by the LengthDelimitedCodec without parallelism.

The third limiting factor pertains to the single Tokio runtime for each node process. Though Tokio engages in work stealing among thread pools, within a single node process, it cannot run parallel tasks exceeding the number of available CPU cores. Running multiple node processes on the same machine entails additional costs for inter-process communication and resource sharing.

The nature of LSB embedding in the existing implementation makes it a sequential process. The `embed_lsb_rgba` function traverses the pixels line by line embedding the encrypted data's bits into the RGBA colors. Parallelism for such an operation would be achieved by dividing the image into regions for concurrent execution, although this doesn't occur in the existing single-threaded implementation.

Scalability could be enhanced through the consideration of the following improvements. Firstly, the current single leader could be replaced by the multi-leader approach such that the coordination of subsets of requests by multiple nodes takes place. Thirdly, the current approach should be improved by batch encoding multiple requests for encryption inside a single RPC call, thus minimizing the effects of the cost of function invocation during the RPC call process. Lastly, the approach could be enhanced through the implementation of image sharding such that the image is divided into pieces encrypted by the followers, thus allowing for parallel execution of the task at the client-side without the need for concurrent execution at the server-side.

8 Conclusion and Next Steps

8.1 Summary of Phase 1 Achievements

The Cloud P2P Environment for Image Sharing project marked a significant milestone in developing a robust, scalable, and secure distributed system during its phase 1. Building on this with the Raft consensus algorithm provided a sound basis for fault tolerance, state replication, and secure cloud-based encryption of images, which was successfully designed and implemented by our team.

In this phase, three load-balancing paradigms were explored and tested: Centralized Leader Execution, Leader Delegation, and Per-Request Election. Of these, after heavy

deliberation, the paradigm of Leader Delegation was used for Phase 1 implementation. In this model, the leadership acts as an executor that delegates encryption and storage to follower nodes, ensuring the execution of the distributed workload, while maintaining cluster-wide consensus.

At the same time, we implemented a complete, workable Graphical User Interface that embeds all the major functionalities: image uploading, initiating encryption, and retrieval of encrypted images. This provided an easy, front-end interface to the Raft cluster for us to test the encryption workflow end-to-end from the user's point of view. The encryption activities performed using the GUI were successful, indicating the proper functioning of the ChaCha20-Poly1305 encryption process and secure embedding of data within images.

The successful completion of these tasks demonstrates that the system meets the Phase 1 objectives, which are to design a fault-tolerant distributed cloud system, integrate secure encryption, perform node replication and synchronization, and provide an interactive client-facing interface for real-world operation.

8.2 Current Limitations

Although the system has proven its core functionality, a number of limitations emerged during testing that will inform Phase 2 development.

One key observation is that GUI-initiated encryption requests are indeed successful in producing the correct encrypted images but also occasionally introduce temporary log inconsistencies across the nodes in a Raft cluster. The encryption itself executes well, but the quick generation and replication of log entries related to encryption lead to sometimes temporary state divergence between nodes. These inconsistencies are transient and would resolve during Raft's next synchronization cycle. However, they expose opportunities for optimizing log commit ordering and the handling of concurrent workloads.

The Leader Delegation mechanism also brought in added communication overhead due to the frequent task assignment and tracking between leader and follower nodes. While this model did improve throughput and fault tolerance, it resulted in more complex coordination compared to a strictly centralized model.

Furthermore, the middleware introduces a single point of failure, which has not been a problem during stress testing, though we seek to find solutions to reduce the possibility of this becoming an issue in future stages of this project.

Finally, in the present architecture, leadership is only changed unless a failure occurs. This design, while enhancing stability, limits the opportunity for exploring scheduled leader rotation, and equitable workload sharing among nodes.

Despite these issues, all the identified limitations can be addressed within the existing Raft-based design, thus setting clear directions for the next phase of development.

8.3 Planned Work for Phase 2

The second phase will extend the system to further refine the system architecture and add new architecture requirements that enhance the system's reliability, scalability, and user experience. In the wake of the comments from Dr. Amr, it is expected that our team will re-enforce the concept of a Centralized Leader Execution model as the core coordination mechanism. However, unlike its former static design, Phase 2 will incorporate the scheduled leader rotation mechanism whereby the leadership automatically switches over

to another node after a scheduled timeout. In other words, the scheduled leader rotation will serve as a mix of the simplistic and deterministic centralized control with the resilient and fair distributed leadership. In addition, several new architecture requirements will be addressed in Phase 2. This includes a Directory of Service for maintaining the mapping of all active peers with their respective roles and current network states in real-time to be updated for easy on-node discovery and connection for inter-node plug and play and fault recovery. Furthermore, the system will move toward more direct peer-to-peer operation, reducing reliance on middleware for coordination and communicating state updates. This shall enhance the efficiency of state and data exchange between nodes. This subphase will also include detailed use-cases, defining and implementing sophisticated interaction flows such as uploads, retrievals, re-encrypt and synchronizations for clients, possible failure paths, and encryption task management policies. Finally, the log inconsistency issue identified in Phase 1 will be resolved by improving Raft's internal commit tracking and introducing concurrency-safe synchronization primitives for multi-threaded GUI-driven encryption. The clients will also be improved with real-time feedback and visualization of the encryption stage and error reporting. Accordingly, this phase will see the system transition from a working proof-of-concept to a complete, operational peer-to-peer cloud environment, with dynamic leadership marking the foundation of a complete distributed service ecosystem in the subsequent ones.